DISTRIBUTED DATABASE SYSTEM FOR A MULTI-BRANCH HOSPITAL MANAGEMENT SYSTEM

Case study

Submitted by

SUKHLEEN KAUR

25MCA20031

In partial fulfilment for the award of the degree of

MASTERS OF COMPUTER APPLICATIONS

IN

UNIVERSITY INSTITUTE OF COMPUTING(UIC)

CHANDIGARH UNIVERSITY

SESSION 2025-27

1. Introduction

A modern hospital organization often operates multiple branches across different cities or regions. Each hospital branch maintains its own database for patient records, doctor information, appointments, medical reports, billing, and pharmacy management.

Maintaining separate databases at each hospital branch creates several challenges:

- **Data Access Limitations:** Patients visiting another branch may not have easy access to their previous medical records.
- **Data Redundancy:** Duplicate patient and doctor records may exist across different branches.
- **Data Inconsistency:** Patient details and medical information may not be updated across all branches.
- **Complex Reporting:** Generating consolidated reports of patients, appointments, revenue, and hospital performance becomes difficult.

To overcome these challenges, the hospital organization intends to integrate all branch databases into a Distributed Database Management System (DDBMS).

This system allows patient, doctor, appointment, and medical records to be accessed and updated across different hospital branches while maintaining data security, reliability, and consistency.

2. Objectives of the Case Study

- To demonstrate how a distributed database can integrate multiple hospital branches.
- To explain data fragmentation, replication, and distributed query processing.
- To provide examples of SQL queries for accessing patient and appointment records across branches.
- To illustrate methods for maintaining data consistency and transaction recovery.
- To discuss performance optimization, security, and concurrency control.
- To design a scalable database architecture that supports the addition of new hospital branches.

3. Why a Distributed Database is Suitable for a Hospital Chain

3.1 Challenges in Centralized Systems

- High network traffic and latency when multiple branches access a single central server.
- Single point of failure: If the central server fails, all hospital branches may experience service interruptions.
- Lack of branch autonomy: Local hospitals may not be able to access important patient information during network failures.
- Difficulty in managing large amounts of medical records from multiple locations.

3.2 Advantages of Distributed Databases

- **Scalability:** New hospital branches can be added easily.
- **Local Autonomy:** Each branch can manage its local appointments, doctors, and patient records.
- **Improved Performance:** Local queries are processed at the nearest hospital database.
- **Fault Tolerance:** Replication provides backup and improves availability.
- **Data Integration:** Doctors can access authorized patient information across different branches.

4. System Architecture Overview

The hospital distributed database system can be designed using a multi-level architecture.

4.1 Node Structure

1. **Regional Nodes:** Each hospital branch maintains its own database containing patient profiles, appointments, doctors, and billing information.
2. **Department Nodes:** Different departments such as cardiology, neurology, emergency, and pharmacy manage their specific operational data.
3. **Central Node:** Maintains global metadata, doctor information, authorized patient records, and consolidated reports.

4.2 Hybrid Architecture

- Combines horizontal fragmentation and vertical fragmentation.
- Patient and appointment records are divided according to hospital branches.
- Sensitive medical and insurance information is stored separately with restricted access.
- Important data such as doctor details and department information can be replicated across branches.

4.3 Data Flow

1. A patient registers at a particular hospital branch.
2. The regional node stores the patient's information.
3. An appointment is created and assigned to an available doctor.
4. Medical records are updated after consultation.
5. Authorized doctors at other branches can retrieve relevant patient information.
6. Transaction logging ensures rollback in case of payment or appointment processing failure.

5. Data Fragmentation Strategy

5.1 Horizontal Fragmentation

- Patient records are divided according to hospital branches.
- Each branch stores the records of patients registered at that location.
- Queries for local patients and appointments become faster.

Example:

- Hospital_North: Patient IDs 1001–2000
- Hospital_South: Patient IDs 2001–3000
- Hospital_East: Patient IDs 3001–4000

5.2 Vertical Fragmentation

- Tables are divided according to different attributes.
- General patient information is separated from sensitive medical and insurance information.

Example:

- Patient_Profile (PatientID, Name, Age, Address)
- Patient_Medical_Info (PatientID, MedicalHistory, Diagnosis, Treatment)

This improves security by restricting access to sensitive patient information.

5.3 Hybrid Fragmentation

- Combines horizontal and vertical fragmentation.
- Patient tables are fragmented horizontally by hospital branch.
- Patient medical records are fragmented vertically according to information type.

6. Data Replication

Data replication ensures that important information is available across different hospital branches.

- **Synchronous Replication:** Critical information is updated immediately across relevant nodes. Example: Updating patient appointment status.
- **Asynchronous Replication:** Less critical information is updated periodically. Example: General department details and non-urgent administrative reports.
- Doctor information and department details can be replicated across hospital branches for faster access.

Benefits

- Minimizes downtime during branch server failures.
- Provides backup for disaster recovery.
- Enables faster access to shared hospital information.
- Improves system availability.

7. Transaction Management and Concurrency Control

7.1 Distributed Transactions

- Operations involving multiple hospital branches can be executed using the Two-Phase Commit (2PC) protocol.
- For example, when a patient books an appointment at another branch, the appointment and doctor availability records must be updated consistently.
- It ensures atomicity: Either all related operations are completed successfully or the entire transaction is rolled back.

7.2 Concurrency Control

- Lock-based mechanisms or timestamp ordering can be used.
- Example: Two patients trying to book the same doctor's last available appointment slot simultaneously.
- Concurrency control ensures that only one booking is confirmed while the other patient receives an unavailable-slot message.

8. Distributed SQL Queries

Example 1: Retrieve all patient records from different hospital branches

```
SELECT * FROM Patients@Hospital_North

UNION ALL

SELECT * FROM Patients@Hospital_South

UNION ALL
```

```
SELECT * FROM Patients@Hospital_East;
```

Example 2: Retrieve the total number of appointments across all branches

```
SELECT SUM(TotalAppointments)
FROM (
   SELECT COUNT(*) AS TotalAppointments
   FROM Appointments@Hospital_North

   UNION ALL

   SELECT COUNT(*)
   FROM Appointments@Hospital_South

   UNION ALL

   SELECT COUNT(*)
   FROM Appointments@Hospital_East
) AS GlobalAppointments;
```

9. Ensuring Data Consistency and Recovery

9.1 Consistency

- Replication protocols ensure that shared hospital information remains synchronized.
- Distributed transactions maintain ACID properties.
- Conflict resolution mechanisms manage simultaneous updates to patient appointments.
- Validation ensures that patient and doctor records remain accurate.

9.2 Recovery

- Each node maintains transaction logs for rollback.
- Periodic checkpoints help restore the database to a consistent state.
- Backup mechanisms protect important patient records.
- Failover mechanisms allow another server to temporarily handle operations when a regional server fails.

10. Security Considerations

- Role-Based Access Control (RBAC) for doctors, nurses, receptionists, and administrators.
- Encryption of sensitive patient and medical information.
- Authentication mechanisms to prevent unauthorized access.
- Audit trails for patient record modifications and financial transactions.
- Multi-Factor Authentication (MFA) for administrative access.
- Regular backups to protect confidential medical information.

11. Performance Considerations

- Load balancing between different hospital database nodes.

- Distributed caching for frequently accessed information.
- Monitoring network latency and database performance.
- Database indexing to improve patient and appointment searches.
- Periodic consolidation of data for hospital performance reports.
- Query optimization to reduce unnecessary communication between branches.

12. Diagrams and Workflows (Placeholders)

The following diagrams can be included:

- Distributed Database Architecture Diagram: Showing the central node and different hospital branches.
- Fragmentation Layout Diagram: Showing horizontal fragmentation of patient records by hospital branch.
- Transaction Flow Diagram: Showing appointment booking and doctor availability updates.
- Replication Flow Diagram: Showing synchronization of shared hospital information.
- Consistency and Recovery Workflow: Showing transaction logging, rollback, and backup mechanisms.

13. Conclusion

Implementing a distributed database system for a multi-branch hospital management system provides an efficient way to manage patient records, doctor information, appointments, and medical data across different locations.

Data fragmentation improves query performance, while replication ensures availability and backup. Transaction management and concurrency control maintain data consistency during appointment booking and other hospital operations.

Security mechanisms such as encryption, role-based access control, and audit trails protect sensitive patient information.

Distributed query processing allows authorized hospital staff to access information across different branches, improving coordination and operational efficiency.

Therefore, a distributed database system provides a reliable, secure, scalable, and efficient solution for managing the information requirements of a modern multi-branch hospital organization.